

□ Как устроен интернет-магазин

Подробный учебник по проекту Amazon Clone

От «Что такое бэкенд?» до «Как 14 модулей работают вместе»

898 тестов · 14 модулей · 1 полный путь покупки

Django + Django REST Framework + PostgreSQL

Для начинающих — объяснено с нуля

□ Оглавление

ЧАСТЬ 1. Базовые понятия

- Урок 1. Бэкенд и фронтенд — две половинки сайта
- Урок 2. HTTP-запросы — как браузер общается с сервером
- Урок 3. JSON — язык обмена данными
- Урок 4. API — меню запросов к бэкенду
- Урок 5. База данных — склад информации
- Урок 6. Django — конструктор для сайтов
- Урок 7. ORM — переводчик Python ↔ SQL
- Урок 8. JWT-токен — электронный паспорт

ЧАСТЬ 2. Как устроен проект

- Урок 9. 14 модулей — отделы магазина
- Урок 10. Анатомия модуля — 7 слоёв одного файла
- Урок 11. Путь запроса — от кнопки до базы и обратно

ЧАСТЬ 3. Модели — таблицы в базе

- Урок 12. BaseModel — общий предок
- Урок 13. Пользователь — модель User
- Урок 14. Товар — модель Product
- Урок 15. Корзина — модели Cart и CartItem
- Урок 16. Заказ — модель Order и статусная машина
- Урок 17. ForeignKey, Constraints, Indexes

ЧАСТЬ 4. Сервисы — мозг проекта

- Урок 18. Service Layer — зачем выносить логику
- Урок 19. transaction.atomic — «Всё или ничего»
- Урок 20. select_for_update — «Занято, не трогай!»
- Урок 21. CartService — читаем реальный код

ЧАСТЬ 5. Сериализаторы, Views, URL

- Урок 22. Сериализатор — переводчик Python ↔ JSON
- Урок 23. View — приёмная запросов
- Урок 24. URL — маршруты
- Урок 25. Permissions — кто может что

ЧАСТЬ 6. Как всё работает вместе

Урок 26. Полный путь покупки — от клика до доставки

Урок 27. Статусная машина заказа

Урок 28. Сигналы — автодействия

Урок 29. Слияние корзин — когда гость логинится

ЧАСТЬ 7. Тесты и миграции

Урок 30. Тесты — 898 автоматических проверок

Урок 31. Миграции — эволюция базы данных

ЧАСТЬ 8. Практические задания

ЧАСТЬ 9. Словарь терминов

ЧАСТЬ 1. Базовые понятия

Прежде чем лезть в код, нужно понять 8 вещей. Без них ничего не будет понятно. Не пропускай!

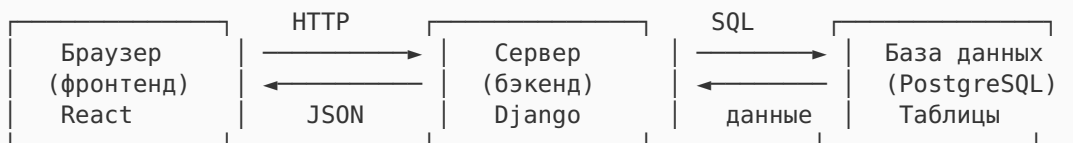
Урок 1. Бэкенд и фронтенд — две половинки сайта

[\[DOC\] Django overview](#) · [React: Quick Start](#)

Когда ты открываешь ozon.ru или wildberries.ru, ты видишь красивые кнопки, картинки, анимации. Это **фронтенд** — программа, которая работает в твоём браузере.

Но когда ты нажимаешь «Купить», браузер отправляет запрос на другой компьютер — **сервер**. Сервер — это **бэкенд**. Он:

- Проверяет, есть ли товар на складе
- Списывает деньги
- Сохраняет заказ в базу данных
- Отправляет email с подтверждением



▣ **Аналогия:** Ресторан. Ты (фронтенд) сидишь за столиком и смотришь в меню. Официант (API) принимает заказ. Кухня (бэкенд) готовит. Склад (база данных) хранит продукты.

▣ **Запомни:** Наш проект — это бэкенд. Он НЕ рисует кнопки. Он только получает запросы и отдаёт данные в формате JSON.

Урок 2. HTTP-запросы — как браузер общается с сервером

[\[DOC\] MDN: HTTP](#) · [HTTP методы](#)

Браузер и сервер общаются через протокол **HTTP** (HyperText Transfer Protocol). Это как язык, на котором они разговаривают.

Есть 4 основных типа запроса:

Тип	Что делает	Аналогия	Пример
GET	Получить данные	Посмотреть меню	Показать корзину
POST	Создать новые данные	Сделать заказ	Добавить товар в корзину
PATCH	Изменить существующие	Изменить блюдо	Поменять количество товара
DELETE	Удалить данные	Отменить заказ	Удалить товар из корзины

Каждый HTTP-запрос получает **ответ с кодом статуса**:

Код	Что значит	Пример
200	Всё ок	Корзина загружена
201	Создано	Товар добавлен в корзину
400	Ошибка в запросе	Передали буквы вместо числа
401	Не авторизован	Нет JWT-токена
403	Доступ запрещён	Обычный юзер лезет в админку
404	Не найдено	Товар не существует
429	Слишком много запросов	Спамшь кнопку
500	Ошибка сервера	Баг в коде

Урок 3. JSON — язык обмена данными

[DOC] [MDN: JSON](#) · [json.org](#)

JSON (JavaScript Object Notation) — это формат, в котором бэкенд отдаёт данные фронтенду. Выглядит так:

```
{
  "id": 1,
  "name": "iPhone 15",
  "price": 89990,
  "in_stock": true,
  "categories": ["Смартфоны", "Электроника"]
}
```

Это просто текст с парами **«ключ: значение»**. Значения могут быть: число, строка (в кавычках), логическое (true/false), массив (в квадратных скобках), объект (в фигурных).

📌 **Аналогия:** JSON — это язык, на котором фронтенд и бэкенд разговаривают между собой. Как английский для дипломатов — оба понимают.

📌 **Задание:** Открой в браузере `http://localhost:8000/api/v1/health/` — ты увидишь JSON-ответ: `{"status": "ok", "version": "1.0.0", "database": "ok"}`

Урок 4. API — меню запросов к бэкенду

[DOC] [Django REST Framework](#) · [REST API](#)

API (Application Programming Interface) — это список всех запросов, которые можно сделать к бэкенду. Как меню в ресторане — ты не идёшь на кухню, ты выбираешь из списка.

Наши основные API-эндпоинты:

Что хочешь	Метод	URL
Войти	POST	/api/v1/auth/login/
Зарегистрироваться	POST	/api/v1/auth/register/
Список товаров	GET	/api/v1/catalog/products/
Карточка товара	GET	/api/v1/catalog/products/iphone-15/
Показать корзину	GET	/api/v1/cart/
Добавить в корзину	POST	/api/v1/cart/items/
Оформить заказ	POST	/api/v1/orders/create/
Мои заказы	GET	/api/v1/orders/
Добавить в избранное	POST	/api/v1/wishlist/add/

□ **Задание:** Открой <http://localhost:8000/api/v1/docs/> — это Swagger, интерактивная документация API. Можно кликать по эндпоинтам и выполнять запросы прямо из браузера!

Урок 5. База данных — склад информации

[\[DOC\] Django Models](#)

База данных (БД) — это место, где хранятся ВСЕ данные магазина. Пользователи, товары, заказы, корзины — всё в таблицах. Как в Excel, но мощнее:

Таблица "users_user":

id	email	username
1	ivan@example.com	ivan
2	maria@mail.ru	maria

Таблица "catalog_product":

id	name	rating
1	iPhone 15	4.5
2	MacBook	4.8

Мы используем **PostgreSQL** — настоящую профессиональную базу данных. Она умеет:

- **Транзакции** — «всё или ничего» (подробнее в уроке 19)
- **Блокировки** — защита от одновременных изменений

- **Индексы** — ускорение поиска
- **Полнотекстовый поиск** — поиск по словам в описаниях

📦 **Аналогия:** База данных — это склад магазина. Товары на полках, записи в журналах. Когда кассир пробивает товар — он ищет его на складе по штрих-коду (id).

Урок 6. Django — конструктор для сайтов

[DOC] [Django Tutorial](#) · [Django Girls](#)

Django (читается «Джэнго») — это фреймворк на Python для создания сайтов.

Фреймворк — это «конструктор» для программиста. Вместо того чтобы писать всё с нуля (обработка запросов, база данных, авторизация), ты берёшь готовые детали и собираешь.

Django предоставляет:

Компонент	Что делает	Аналогия
ORM	Работа с базой через Python	Переводчик
Admin	Готовая админка	Панель управления
Auth	Регистрация, логин, пароли	Охранник
DRF	Создание API (REST Framework)	Официант

Урок 7. ORM — переводчик Python ↔ SQL

[DOC] [Django ORM Queries](#)

ORM (Object-Relational Mapping) — это способ работать с базой данных через Python-объекты, без написания SQL.

Вместо:

```
SELECT * FROM catalog_product WHERE rating > 4.0 ORDER BY min_price;
```

Пишешь:


```
Product.objects.filter(rating__gt=4.0).order_by('min_price')
```

ORM сам переводит Python-код в SQL и отправляет в базу.

□ **Аналогия:** ORM — это переводчик. Ты говоришь по-русски, ORM переводит на SQL (язык базы данных).

□ **Запомни:** Два подчёркивания `__` в ORM — это «особый вызов»: `__gt` = greater than, `__lt` = less than, `__icontains` = поиск без учёта регистра.

Урок 8. JWT-токен — электронный паспорт

[DOC] SimpleJWT · jwt.io

Когда ты логишься, бэкенд выдаёт тебе **JWT-токен** — длинную строку вроде паспорта:

```
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ1c2VyX2lkIjoxfQ.abc123...
```

Этот токен фронтенд отправляет с каждым запросом, и бэкенд понимает: «А, это Иван!»

Два токена:

Токен	Живёт	Зачем
Access	15 минут	Для обычных запросов
Refresh	7 дней	Получить новый access без повторного логина

□ **Аналогия:** JWT — это браслет в аквапарке. Показал на входе — пустили. Потерял — иди на кассу (логинься заново). Действует до конца дня (15 мин). Если просрочился — показываешь специальный талон (refresh) и получаешь новый браслет.

□ **Запомни:** Токен хранится в `localStorage` в браузере и отправляется в заголовке `Authorization: Bearer <токен>` с каждым запросом.

ЧАСТЬ 2. Как устроен проект

Урок 9. 14 модулей — отделы магазина

[\[DOC\] Django Apps](#)

Наш проект состоит из **14 модулей** (папок). Каждый модуль — это отдельная «деталь» магазина:

□ **core** — Фундамент. BaseModel — общий предок всех моделей (created_at, updated_at)

□ **users** — Пользователи. Регистрация, логин, профиль, адреса доставки

□ **catalog** — Каталог. Товары, категории, бренды, теги, варианты, изображения

□ **pricing** — Цены. Цена, скидочная цена, история изменений цен

□ **cart** — Корзина. Добавить, убрать, изменить количество, слить гостевую

□ **orders** — Заказы. Оформить, отменить, статусы (PENDING → DELIVERED)

□ **inventory** — Склад. Остатки, резервы, движения товаров

 **payments** — Оплата. Платежи, возвраты, вебхуки от платёжных систем

 **reviews** — Отзывы. Рейтинг 1-5, текст, фото, подтверждённая покупка

 **discounts** — Скидки. Купоны, промокоды, кампании (% или фиксированная сумма)

 **shipping** — Доставка. Зоны, методы, отслеживание посылок

 **wishlist** — Избранное. Сохранить на потом, переместить в корзину

 **notifications** — Уведомления. «Заказ отправлен», «Оплата прошла»

 **analytics** — Аналитика. Просмотры, продажи, конверсия, дашборд

 **Аналогия:** Каждый модуль — это отдел в магазине. Отдел электроники, отдел одежды, касса, склад, бухгалтерия. Каждый делает свою работу, но все работают вместе.

Урок 10. Анатомия модуля — 7 слоёв

[\[DOC\] Service Layer Pattern](#)

Каждый модуль устроен одинаково. Возьмём для примера корзину (`apps/cart/`):

Папка/файл	Что делает	Аналогия
<code>models/</code>	Таблицы в базе данных	Полки на складе
<code>services/</code>	Бизнес-логика (мозг)	Кухня (готовит по рецепту)
<code>serializers/</code>	Переводят объекты в JSON и обратно	Таможня (проверяет и упаковывает)
<code>api_views/</code>	Принимают HTTP-запросы	Официант (принимает заказ)
<code>urls.py</code>	Маршруты (URL → View)	Таблички на дверях
<code>tests/</code>	Проверяют что всё работает	Техосмотр
<code>migrations/</code>	История изменений базы	Чертежи строительства

Урок 11. Путь запроса — от кнопки до базы и обратно

[\[DOC\] Django Request/Response](#)

Когда фронтенд отправляет `POST /api/v1/cart/items/ {variant_id: 5, quantity: 2}`:

1. URL: `/api/v1/cart/items/` → `CartItemView` (`api_views/`)
2. View: Принимает запрос, вызывает сериализатор
3. Serial: Проверяет данные (`variant_id` есть? `quantity > 0`?)
4. View: Вызывает `CartService.add_item()` (`services/`)
5. Service: `@transaction.atomic + select_for_update`
Проверяет сток, ищет корзину, создаёт `CartItem`
6. ORM: `INSERT INTO cart_cartitem (...)` (`models/`)
7. Service: Возвращает обновлённую корзину
8. View: Отдаёт JSON-ответ (`serializers/`)

Аналогия: Как в ресторане: официант (URL) принимает заказ → проверяет бланк (Serializer) → передаёт на кухню (Service) → повар готовит по рецепту (бизнес-логика) → берёт продукты из холодильника (ORM → БД) → отдаёт блюдо через официанта (View → JSON).

□ **Запомни:** Главное правило: **View НЕ содержит бизнес-логику!**
View только принимает запрос, валидирует, вызывает сервис, возвращает ответ. Вся «умная» работа — в сервисе.

ЧАСТЬ 3. Модели — таблицы в базе

Урок 12. BaseModel — общий предок

[\[DOC\] Abstract base classes](#)

У нас есть абстрактная модель `apps/core/models/base_model.py`:

```
class BaseModel(models.Model):
    created_at = models.DateTimeField(auto_now_add=True) # Один раз при
    создании
    updated_at = models.DateTimeField(auto_now=True)    # Каждый раз
    при save()

    class Meta:
        abstract = True # Django НЕ создаёт таблицу для BaseModel!
```

Все модели (кроме User) наследуют BaseModel → получают `created_at` и `updated_at` бесплатно.

□ **Аналогия:** BaseModel — как заготовка для паспорта. Все паспорта имеют страницу «Дата выдачи» и «Дата замены». Не нужно каждый раз рисовать эти страницы — они уже есть в шаблоне.

⚠ **Важно:** `abstract = True` — критически важно! Без него Django создал бы отдельную таблицу `core_basemodel`, а все модели ссылались бы на неё через `OneToOneField` → лишние JOIN'ы → медленные запросы.

Урок 13. Пользователь — модель User

[\[DOC\] Custom User model](#)

Модель User — особенная: она наследует **AbstractUser** (не BaseModel!):

```
class User(AbstractUser):
    phone = models.CharField(max_length=20, blank=True, default='')
    email = models.EmailField(unique=True) # Обязательный, уникальный!
```

AbstractUser уже даёт: username, email, first_name, last_name, is_active, is_staff, is_superuser, date_joined, password (хэшированный).

Мы переопределили email: в стандартном AbstractUser он blank=True (необязательный), а мы сделали unique=True (обязательный, уникальный).

⚠ **Важно:** User НЕ наследует BaseModel → у него НЕТ created_at и updated_at. Зато есть date_joined от AbstractUser.

Урок 14. Товар — модель Product

[\[DOC\] SlugField](#) · [django-treebeard](#)

Товар — центральная сущность магазина. Ключевые поля:

```
class Product(BaseModel):
    uuid = models.UUIDField(default=uuid.uuid4, editable=False)
    slug = models.SlugField(unique=True)
    name = models.CharField(max_length=500)
    description = models.TextField(blank=True)
    status = models.CharField(choices=ProductStatus.choices,
default=ProductStatus.DRAFT)
    brand = models.ForeignKey(Brand, on_delete=models.SET_NULL,
null=True)
    min_price = models.DecimalField(max_digits=12, decimal_places=2,
default=0)
    max_price = models.DecimalField(max_digits=12, decimal_places=2,
default=0)
    rating = models.DecimalField(max_digits=3, decimal_places=2,
default=0)
    reviews_count = models.PositiveIntegerField(default=0)
    views_count = models.PositiveIntegerField(default=0)
```

Зачем uuid вместо id? Если показывать id=1, id=2, id=3 — конкурент узнает, сколько у тебя товаров. UUID непредсказуем.

Зачем slug? URL /products/iphone-15/ красивее и лучше для SEO, чем /products/550e8400-e29b/.

Зачем min_price/max_price? Денормализованные цены — чтобы не делать JOIN к Price при каждом показе списка товаров.

📌 **Запомни: Денормализация** — когда мы храним копию данных в другой таблице для скорости. Например, min_price в Product — это копия минимальной цены из Price. При изменении цены нужно обновить и Product.min_price (через сигнал).

Урок 15. Корзина — модели Cart и CartItem

[\[DOC\] ForeignKey](#)

```
class Cart(BaseModel):
    user = models.ForeignKey(User, on_delete=models.CASCADE, null=True)
    session_key_hash = models.CharField(max_length=64, null=True)
    is_active = models.BooleanField(default=True)
```

Два типа владельца:

- **Авторизованный:** user заполнен, session_key_hash = NULL
- **Гость:** user = NULL, session_key_hash = SHA-256 хэш ключа сессии

⚠ **Важно:** Почему хэш, а не оригинальный session_key? Если база утечёт — хакер получит хэши, но не сможет подставить их в cookie. SHA-256 необратим!

```
class CartItem(BaseModel):
    cart = models.ForeignKey(Cart, on_delete=models.CASCADE,
                             related_name='items')
    variant = models.ForeignKey(ProductVariant, on_delete=models.CASCADE)
    quantity = models.PositiveIntegerField(default=1)
```

Один CartItem = одна позиция в корзине. related_name='items' позволяет писать cart.items.all().

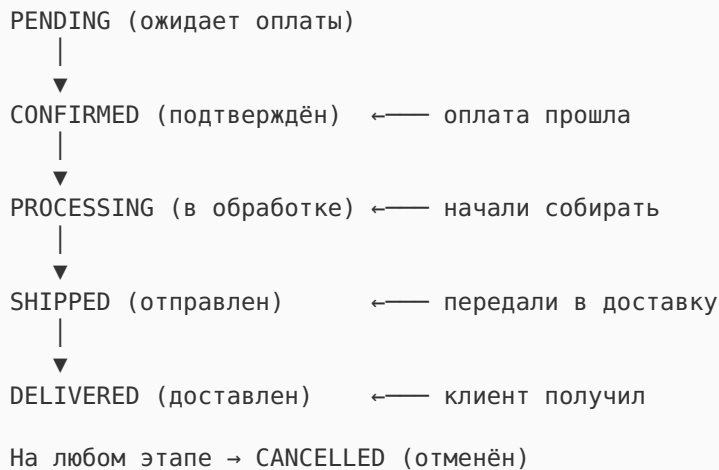
Урок 16. Заказ — модель Order и статусная машина

[\[DOC\] django-fsm](#)

Заказ — одна из самых сложных моделей. Ключевые поля:

```
class Order(BaseModel):
    status = models.CharField(choices=OrderStatus.choices,
                              default=OrderStatus.PENDING)
    user = models.ForeignKey(User, on_delete=models.PROTECT)
    order_number = models.CharField(max_length=20, unique=True,
                                    editable=False)
    subtotal = models.DecimalField(max_digits=12, decimal_places=2)
    delivery_cost = models.DecimalField(max_digits=10, decimal_places=2,
                                        default=0)
    discount = models.DecimalField(max_digits=12, decimal_places=2,
                                   default=0)
    total = models.DecimalField(max_digits=12, decimal_places=2)
```

Статусы заказа (FSM — Finite State Machine, конечный автомат):



Правила переходов зашифрованы в словаре:

```
ORDER_STATUS_TRANSITIONS = {
    'PENDING': ['CONFIRMED', 'CANCELLED'],
    'CONFIRMED': ['PROCESSING', 'CANCELLED'],
    'PROCESSING': ['SHIPPED', 'CANCELLED'],
    'SHIPPED': ['DELIVERED', 'CANCELLED'],
    'DELIVERED': [], # Терминальный — никуда больше
    'CANCELLED': [], # Терминальный — никуда больше
}
```

□ **Запомни:** Нельзя перескочить из PENDING сразу в SHIPPED — сервис не позволит. Это как уровни в игре: нельзя перейти на уровень 3, не пройдя уровень 2.

Адрес — SNAPSHOT (копия), не FK! Когда пользователь меняет адрес в профиле, данные заказа НЕ меняются. Это критично для чеков и возвратов.

DecimalField, не float! `0.1 + 0.2 = 0.30000000000000004` в float. Decimal считает точно до копеек.

Урок 17. ForeignKey, Constraints, Indexes

[\[DOC\] Model Meta options](#) · [Constraints](#)

ForeignKey — связь между таблицами

```
CartItem.cart = models.ForeignKey(Cart, on_delete=models.CASCADE,
related_name='items')
```

Это значит: каждый CartItem привязан к ОДНОЙ корзине, а корзина имеет МНОГО items.

```
Cart #1 ← CartItem (iPhone, 2 шт)
        ← CartItem (MacBook, 1 шт)

Cart #2 ← CartItem (AirPods, 3 шт)
```

Constraints — правила в базе

```
models.UniqueConstraint(
    fields=['user'],
    condition=Q(is_active=True) & Q(user__isnull=False),
    name='unique_active_user_cart',
)
```

Это правило: у одного пользователя — максимум ОДНА активная корзина. База данных сама это гарантирует.

Indexes — ускорение поиска

```
models.Index(fields=['user', 'is_active'], name='cart_user_active_idx')
```

Индекс — как алфавитный указатель в книге. Без него поиск «все активные корзины пользователя» = перелистать ВСЮ книгу (Full Table Scan). С индексом — сразу на нужную страницу.

ЧАСТЬ 4. Сервисы — мозг проекта

Урок 18. Service Layer — зачем выносить логику

[DOC] [Cosmic Python](#) · [Django transactions](#)

Зачем сервисы отдельным слоем? Представь:

- БЕЗ сервисов (логика в View):
 - View сам проверяет сток, создаёт корзину, считает цену, резервирует товар
 - 200 строк в одном методе
 - если нужно «создать заказ из CLI» — дублируем код
- С СЕРВИСАМИ:
 - View: «CartService, добавь товар» — 3 строки
 - Service: проверяет сток, создаёт корзину, считает цену — всё в одном месте
 - View, CLI, management-команда используют один сервис

□ **Запомни:** Правило: **View** → **Serializer** → **Service** → **ORM**. View не знает про транзакции. Serializer не знает про бизнес-логику. Service не знает про HTTP.

Урок 19. transaction.atomic — «Всё или ничего»

[DOC] [atomic\(\)](#)

```
@transaction.atomic
def transfer_money(from_account, to_account, amount):
    from_account.balance -= amount      # Шаг 1
    from_account.save()
    to_account.balance += amount        # Шаг 2
    to_account.save()
    # Если шаг 2 упал — шаг 1 тоже откатывается автоматически!
```

□ **Аналогия:** transaction.atomic — как эскроу-счёт при покупке квартиры. Или деньги переведены покупателю, или квартира — продавцу. Не бывает «деньги списаны, а квартира не передана».

Урок 20. select_for_update — «Занято, не трогай!»

[DOC] [select_for_update\(\)](#)

```
stock = Stock.objects.select_for_update().get(variant=variant)
stock.quantity -= 1
stock.save()
```

Пока мы держим `select_for_update()`, другой запрос НЕ может изменить эту строку. Это защищает от **race condition** — когда два человека одновременно покупают последний товар.

```
БЕЗ select_for_update:
Запрос A: stock.quantity = 1 ← видит 1
Запрос B: stock.quantity = 1 ← тоже видит 1
Запрос A: quantity -= 1 → 0 ← сохраняет 0
Запрос B: quantity -= 1 → 0 ← тоже сохраняет 0
Результат: продали 2 товара, а был только 1! □

С select_for_update:
Запрос A: SELECT FOR UPDATE → quantity = 1 ← ЗАБЛОКИРОВАЛ
Запрос B: SELECT FOR UPDATE → ЖДЁТ...
Запрос A: quantity -= 1 → 0, COMMIT ← разблокировал
Запрос B: SELECT FOR UPDATE → quantity = 0 ← видит 0
Запрос B: ОШИБКА «Недостаточно на складе» □
```

Урок 21. CartService — читаем реальный код

[DOC] [F\(\) expressions](#)

Вот как выглядит добавление товара в корзину (упрощённо):

```
@staticmethod
@transaction.atomic
def add_item(cart, variant_id, quantity=1):
    # Шаг 0: Базовая валидация
    if quantity < 1:
        raise ValidationError('Количество должно быть ≥ 1')

    # Шаг 1: Проверка лимита позиций
    if cart.items.count() >= 100:
        raise ValidationError('Слишком много товаров в корзине')

    # Шаг 2: Загрузка варианта с проверкой активности
    variant = ProductVariant.objects.select_related('product', 'stock')\
        .get(pk=variant_id, is_active=True)

    # Шаг 3: Проверка что ТОВАР тоже активен
    if variant.product.status != 'active':
        raise NotFound('Товар недоступен для заказа.')

    # Шаг 4: select_for_update – блокировка строки
    existing = CartItem.objects.select_for_update()\
        .filter(cart=cart, variant=variant).first()

    # Шаг 5: Проверка остатков на складе
    stock = getattr(variant, 'stock', None)
    if stock and quantity > stock.quantity:
        raise ValidationError('Недостаточно на складе')

    # Шаг 6: Создание или обновление
    if existing:
        existing.quantity += quantity
        existing.save()
    else:
        CartItem.objects.create(cart=cart, variant=variant,
                                quantity=quantity)

    return cart
```

6 шагов! Каждый — защита от определённой ошибки. Без любого из них — баг:

Шаг	Что проверяет	Что будет без него
0	quantity ≥ 1	Добавят -5 товаров
1	Лимит 100 позиций	100000 позиций → тормозит
2	Вариант активен	Купят удалённый товар
3	Товар активен	Купят черновик
4	select_for_update	Race condition — 2 позиции вместо 1
5	Остаток на складе	Продадут то, чего нет

ЧАСТЬ 5. Сериализаторы, Views, URL

Урок 22. Сериализатор — переводчик Python ↔ JSON

[\[DOC\] DRF Serializers](#)

Сериализатор делает две вещи:

1. **Вход:** проверяет данные из запроса (валидация)
2. **Выход:** превращает Python-объект в JSON

Пример входного сериализатора:

```
class AddToCartInputSerializer(serializers.Serializer):
    variant_id = serializers.IntegerField(min_value=1)    # Обязательное,
    ≥ 1
    quantity = serializers.IntegerField(min_value=1, default=1) #
    Опциональное, по умолчанию 1
```

Если отправить `{"variant_id": "abc"}` → ошибка: «variant_id должен быть числом».

Пример выходного сериализатора:

```
class CartItemSerializer(serializers.ModelSerializer):
    class Meta:
        model = CartItem
        fields = ['id', 'product_name', 'sku', 'price', 'quantity',
        'total_price']
```

Результат:

```
{
  "id": 5,
  "product_name": "iPhone 15",
  "sku": "IP15-128-BLK",
  "price": "89990.00",
  "quantity": 2,
  "total_price": "179980.00"
}
```

📌 **Аналогия:** Сериализатор — таможенник. Проверяет багаж (данные) на входе и упаковывает товар (объект) для отправки на выходе.

Урок 23. View — приёмная запросов

[\[DOC\] DRF APIView](#)

View — это класс, который принимает HTTP-запрос и возвращает HTTP-ответ:

```
class CartItemView(APIView):
    permission_classes = (AllowAny,) # Любой может добавить в корзину

    def post(self, request):
        # 1. Валидируем входящие данные
        input_ser = AddToCartInputSerializer(data=request.data)
        input_ser.is_valid(raise_exception=True)

        # 2. Получаем корзину (создаём если нет)
        cart = CartService.get_or_create_cart(request)

        # 3. Вызываем бизнес-логику
        CartService.add_item(
            cart=cart,
            variant_id=input_ser.validated_data['variant_id'],
            quantity=input_ser.validated_data.get('quantity', 1),
        )

        # 4. Возвращаем обновлённую корзину
        return Response(CartSerializer(cart).data, status=201)
```

📌 **Запомни:** View НЕ содержит бизнес-логику. Только 4 шага: принял → валидировал → вызвал сервис → вернул ответ.

Урок 24. URL — маршруты

[\[DOC\] Django URL dispatcher](#)

URL-файл связывает адрес с View:

```
urlpatterns = [  
    path('', CartView.as_view(), name='cart'),  
    path('items/', CartItemView.as_view(), name='cart-items'),  
    path('items/<int:item_id>/', CartItemDetailView.as_view()),  
    path('merge/', CartMergeView.as_view(), name='cart-merge'),  
]
```

В `config/urls.py` всё собирается: `path('api/v1/cart/',
include('apps.cart.urls'))`

Итого полный URL: `/api/v1/cart/items/`

Урок 25. Permissions — кто может что

[\[DOC\] DRF Permissions](#)

Permission	Кто имеет доступ	Пример
AllowAny	Любой (даже без токена)	Каталог, корзина
IsAuthenticated	Только с JWT-токеном	Оформление заказа
IsAdminUser	Только админ (<code>is_staff=True</code>)	Аналитика

ЧАСТЬ 6. Как всё работает вместе

Урок 26. Полный путь покупки

[DOC] [django-cors-headers](#)

Вот что происходит, когда пользователь покупает товар:

1. ☐ ПОЛЬЗОВАТЕЛЬ ЗАХОДИТ НА САЙТ
→ users: Регистрация/Логин → JWT-токен
2. ☐ ПРОСМАТРИВАЕТ КАТАЛОГ
→ catalog: Список товаров, категории, бренды
→ analytics: Записываем просмотр (record_view)
3. ☐ КЛАДЁТ В КОРЗИНУ
→ cart: CartService.add_item()
→ inventory: Проверяем остаток (stock.quantity)
4. ☐ ОФОРМЛЯЕТ ЗАКАЗ
→ orders: OrderService.create_from_cart()
→ cart: Корзина деактивируется
→ inventory: Статус CONFIRMED → резервируем товар
→ discounts: Применяем купон если есть
→ shipping: Рассчитываем стоимость доставки
→ notifications: «Ваш заказ создан!»
5. ☐ ОПЛАЧИВАЕТ
→ payments: PaymentService.process()
→ orders: Статус CONFIRMED → PROCESSING
→ notifications: «Оплата прошла!»
6. ☐ ОТПРАВЛЯЕТСЯ
→ shipping: Shipment → IN_TRANSIT
→ orders: Статус PROCESSING → SHIPPED
→ notifications: «Заказ отправлен!»
7. ☐ ДОСТАВЛЯЕТСЯ
→ shipping: Shipment → DELIVERED
→ orders: Статус SHIPPED → DELIVERED
→ inventory: Списываем со склада (commit)
→ notifications: «Заказ доставлен!»
8. ☐ ОСТАВЛЯЕТ ОТЗЫВ
→ reviews: ReviewService.create()
→ catalog: Пересчитываем рейтинг товара

Урок 27. Статусная машина заказа (подробнее)

[DOC] [django-fsm](#)

Статусная машина гарантирует, что заказ проходит через статусы по правилам. Сервис `OrderService.transition_status()` проверяет:

```
def transition_status(order, new_status):
    allowed = ORDER_STATUS_TRANSITIONS.get(order.status, [])
    if new_status not in allowed:
        raise ValidationError(
            f'Нельзя перейти из {order.status} в {new_status}'
        )
    order.status = new_status
    order.save()
```

Это защищает от багов: никто не сможет «перепрыгнуть» статус или вернуть доставленный заказ обратно в обработку.

Урок 28. Сигналы — автодействия

[\[DOC\] Django Signals](#)

Сигнал — автоматическое действие при каком-то событии. Django сообщает: «Произошло X!» — и твой код реагирует.

Пример: при создании пользователя автоматически создаём профиль:

```
@receiver(post_save, sender=User)
def create_user_profile(sender, instance, created, **kwargs):
    if created:
        UserProfile.objects.create(user=instance)
```

Пример: при создании отзыва пересчитываем рейтинг товара:

```
@receiver(post_save, sender=Review)
def recalculate_product_rating(sender, instance, **kwargs):
    product = instance.product
    reviews = Review.objects.filter(product=product, is_approved=True)
    product.rating = reviews.aggregate(avg=Avg('rating'))['avg'] or 0
    product.reviews_count = reviews.count()
    product.save()
```

□ **Аналогия:** Сигнал — как датчик дыма. Сработал → автоматически вызвал пожарных. Ты не звонишь сам — система реагирует сама.

Урок 29. Слияние корзин — когда гость логинится

[DOC] [Session framework](#)

Гость наполнил корзину. Потом залогинился. Что делать с гостевой корзиной? **Слить** её в пользовательскую!

Гостевая корзина:

iPhone 15 × 2

AirPods × 1

Пользовательская корзина:

MacBook × 1

РЕЗУЛЬТАТ СЛИЯНИЯ:

iPhone 15 × 2 ← из гостевой

MacBook × 1 ← было

AirPods × 1 ← из гостевой

Алгоритм `CartService.merge_guest_into_user_cart()` :

1. Найти гостевую корзину по `session_key_hash`
2. Получить/создать корзину пользователя
3. Для каждой позиции гостевой корзины:
 - Проверить `variant.is_active` → пропустить неактивные
 - Если позиция уже есть в юзерской → сложить `quantity`
 - Иначе → создать новую позицию
 - Ограничить `quantity` доступным остатком (`stock`)
4. Деактивировать гостевую корзину (НЕ удалять!)

⚠ **Важно:** Гостевую корзину мы ДЕАКТИВИРУЕМ, а не удаляем — для аналитики (сколько корзин было слито) и восстановления.

ЧАСТЬ 7. Тесты и миграции

Урок 30. Тесты — 898 автоматических проверок

[DOC] [Django Testing](#) · [DRF Testing](#)

Тесты проверяют, что код работает правильно. У нас **898 тестов** — и все проходят!

```
class CartServiceTests(TestCase):
    def test_add_item(self):
        """Добавление товара в корзину."""
        cart = self._create_cart()
        CartService.add_item(cart=cart, variant_id=self.variant.pk,
quantity=2)
        self.assertEqual(cart.items.count(), 1)          # 1 позиция
        self.assertEqual(cart.items.first().quantity, 2) # количество = 2

    def test_add_same_item_increments(self):
        """Повторное добавление увеличивает количество."""
        cart = self._create_cart()
        CartService.add_item(cart=cart, variant_id=self.variant.pk,
quantity=1)
        CartService.add_item(cart=cart, variant_id=self.variant.pk,
quantity=3)
        self.assertEqual(cart.items.count(), 1)          # всё ещё 1
позиция
        self.assertEqual(cart.items.first().quantity, 4) # 1 + 3 = 4
```

□ **Аналогия:** Тесты — как техосмотр автомобиля. Перед выездом на дорогу проверяем: тормоза работают? фары горят? руль крутится? Если хоть одна проверка не прошла — на дорогу нельзя.

Запуск:


```
python manage.py test          # Все 898 тестов
python manage.py test apps.cart # Только корзину
python manage.py test apps.orders.tests.test_services # Только сервисы заказов
```

Урок 31. Миграции — эволюция базы данных

[\[DOC\] Django Migrations](#)

Миграция — это инструкция «как изменить базу данных». Как коммиты в Git, но для структуры таблиц.

```
# 0001_initial.py — создаёт таблицу
operations = [
    migrations.CreateModel(
        name='Cart',
        fields=[
            ('id', models.BigAutoField(primary_key=True)),
            ('user', models.ForeignKey(...)),
            ('is_active', models.BooleanField(default=True)),
        ],
    ),
]
```

Команды:

```
python manage.py makemigrations # Модель → SQL (создать миграцию)
python manage.py migrate       # Выполнить SQL (применить миграцию)
```

📌 **Аналогия:** Миграции — как строительные чертежи. Сначала «Фундамент» (0001), потом «Стены» (0002), потом «Крыша» (0003). Нельзя построить крышу без стен!

⚠ **Важно:** Никогда не удаляй миграции, которые уже применены на сервере! Это как сжечь чертежи, когда дом уже построен. Если нужно откатить — используй `python manage.py migrate app_name migration_number .`

ЧАСТЬ 8. Практические задания

□ Задание 1: Создай товар через Django Admin

1. Запусти сервер: `python manage.py runserver`
2. Открой `http://localhost:8000/admin/`
3. Залогинься (создай superuser: `python manage.py createsuperuser`)
4. Создай бренд, категорию, товар, вариант товара

Что ты поймёшь: Как данные попадают в базу через админку.

□ Задание 2: Получи данные через API

1. Залогинься через API (curl или Postman)
2. Скопируй access-токен
3. Получи список товаров: `GET /api/v1/catalog/products/`

Что ты поймёшь: Как фронтенд общается с бэкендом.

□ Задание 3: Добавь товар в корзину через API

```
POST /api/v1/cart/items/  
Authorization: Bearer <токен>  
{  
  "variant_id": 1,  
  "quantity": 2  
}
```

Потом посмотри корзину: `GET /api/v1/cart/`

□ Задание 4: Напиши свой тест

Добавь в `apps/cart/tests/test_services.py` :

```
def test_add_item_quantity_zero_raises_error(self):
    """Нельзя добавить 0 товаров."""
    cart = self._create_cart()
    with self.assertRaises(ValidationError):
        CartService.add_item(
            cart=cart,
            variant_id=self.variant_a.pk,
            quantity=0,
        )
```

Запусти: `python manage.py test apps.cart`

□ Задание 5: Прочитай код модуля wishlist

Выбери модуль `apps/wishlist/` (он маленький) и прочти ВСЕ файлы по порядку:

1. `models/wishlist.py` — какие поля?
2. `services/wishlist_service.py` — какие методы?
3. `serializers/wishlist_serializers.py` — какие поля в JSON?
4. `api_views/wishlist_views.py` — какие URL?
5. `urls.py` — какие маршруты?
6. `tests/test_services.py` — какие проверки?

□ Задание 6: Нарисуй путь заказа

Нарисуй на бумаге полный путь покупки (урок 26). Подпиши, какой модуль за что отвечает. Попробуй объяснить вслух.

ЧАСТЬ 9. Словарь терминов

Термин	Простыми словами
Бэкенд	Программа на сервере (не видна пользователю)
Фронтенд	Программа в браузере (кнопки, картинки)
API	Меню запросов к бэкенду
JSON	Формат данных (текст с ключами и значениями)
HTTP	Протокол общения браузера и сервера
ORM	Переводчик Python ↔ SQL
Модель	Таблица в базе данных
ForeignKey	Ссылка на другую таблицу
Сериализатор	Переводчик Python-объект ↔ JSON
View	Обработчик HTTP-запроса
Сервис	Бизнес-логика (мозг)
Migration	Инструкция по изменению базы
Тест	Автоматическая проверка кода
JWT	Электронный паспорт пользователя
Transaction	«Всё или ничего» — группа операций
select_for_update	«Занято, не трогай!» — блокировка строки
Signal	Автодействие при событии
Constraint	Правило в базе (уникальность, проверка)
Index	Ускорение поиска (как алфавитный указатель)
FSM	Конечный автомат (статусная машина)
CORS	Разрешение для фронтенда делать запросы
Throttle	Ограничение количества запросов
Denormalization	Копия данных для скорости
Race condition	Баг при одновременных запросах
IDOR	Уязвимость: доступ к чужим данным по ID

Snapshot

Замороженная копия данных (адрес в заказе)
